

Mess Management System — Software Requirements Specification

Sep 24, 2026 · @Zyad

Structured to the ISO/IEC/IEEE 29148 SRS outline. Version 2.0 supersedes the earlier free-form SRS and is the governing reference for all design and build decisions.

1. Introduction

1.1 Purpose

This SRS specifies the requirements for the Mess Management System (mess-proj): a browser-based room-status and fault-ticketing system for Defence Forces messes. It is the single governing reference for what the system shall do. Requirements are stated with “shall”, carry a unique identifier, and are individually verifiable (Section 7).

1.2 Scope

The system shall let mess staff record which rooms are available, occupied, or under maintenance; log and resolve faults against rooms; and give brigade-level users a read-only, cross-mess summary.

Release	Boundary
Prototype (v1)	Single server, SQLite, fictional seed data, one or more messes on a local network.
Rollout (target)	Multi-site deployment across additional messes. Network and hosting architecture to be determined following organisational approval (REG-5).

Out of scope for v1 Prototype: native mobile or desktop clients; integration with other DF systems; self-service user registration. **Deferred beyond the prototype:** room-status history table, containerization, scheduled reporting. **Excluded by decision:** event streaming.

1.3 Intended Audience and Document Use

Audience	Use of this document
Developer (Zyad)	Build against Sections 3–6; check every change against the requirement it serves
Tester(Zyad) / reviewer(TBC)	Derive tests from Section 7 and the traceability matrix (Appendix C)
Mess secretaries and workers	Describe Sections 2.2, 2.3 and Appendix B
Information-systems / accreditation reviewers	Assess Sections 4.2, 6.1, 6.2 for accreditation and deployment approval
Recruiters and technical reviewers	Read Sections 1–2 and Appendix E for scope and design judgement

1.4 References

Ref	Document	Version
[R1]	mess-system-build-roadmap.md	Current project file
[R2]	Mess Management System Study Roadmap	1.0, Aug 2026
[R3]	ISO/IEC/IEEE 29148 SRS example template, well-architected-guide.com	Feb 2025
[R4]	OWASP Top Ten and Cheat Sheet Series	Current
[R5]	NIST SP 800-63B, §5.1.1 Memorised Secrets	Current
[R6]	Regulation (EU) 2016/679 (GDPR)	2016
[R7]	Data Protection Act 2018 (Ireland)	2018
[R8]	SQLite documentation (Foreign Keys, WAL, Datatypes, Query Planning)	Current
[R9]	PEP 249, Python DB-API 2.0	2.0

1.5 Definitions, Acronyms, and Abbreviations

Key terms are below; the full glossary is Appendix A.

Term	Meaning
Mess	An accommodation unit within a site (officers', NCOs', privates')
Site	A barracks or camp containing one or more messes
Scope	The set of messes a user may read or write
Scope resolver	The single function that computes a user's scope (FR-2.2)
Ticket	A fault report logged against a room
DF / LAN	Defence Forces / Local Area Network
CSRF / XSS / SQLi	Cross-Site Request Forgery / Cross-Site Scripting / SQL Injection
WAL	Write-Ahead Logging, an SQLite journal mode
MoSCoW	Must / Should / Could / Won't priority scale

1.6 Document Overview

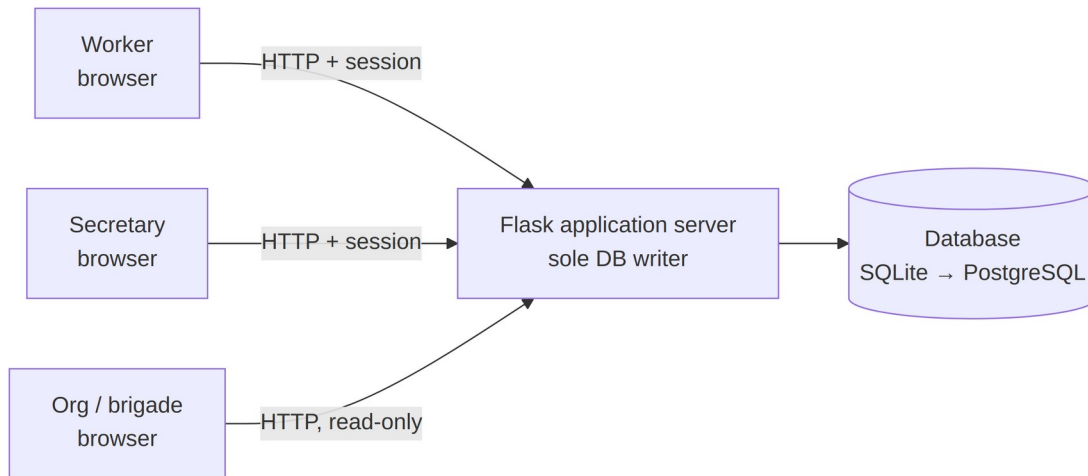
Section 2 describes the product and its context. Section 3 holds functional requirements; Section 4 non-functional; Section 5 interfaces; Section 6 legal, deployment, localization, and data. Section 7 defines how each requirement is verified and accepted. Appendices cover glossary, use cases, traceability, diagrams, open issues, and revision history.

Requirement ID scheme: FR-n.m (functional), NFR-PERF/SEC/REL/USE/MNT/PRT/QA-n (non-functional), IF-n (interface), REG/DEP/LOC/DAT-n (other), CON-n (constraint). **Priority:** MoSCoW, relative to the prototype release.

2. Overall Description

2.1 System Perspective

mess-proj is a new, self-contained system. It replaces informal, location-bound record keeping and does not integrate with any existing DF system. It is built independently from public documentation only.



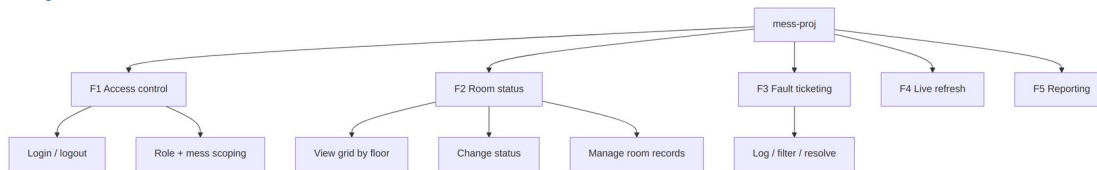
Context diagram: Worker, Secretary and Org browsers connect over HTTP with session to the Flask application server, the sole database writer, which connects to the database

All clients talk only to the server; only the server opens the database. The three tiers carry different trust levels:

Tier	Runs on	Responsibility	Trust
Presentation	Browser on client	Render HTML, capture input, poll for updates	Untrusted
Application	Flask server process	Authenticate, authorize, validate, apply business rules	Trusted
Data	Database	Persist state; enforce keys, NOT NULL, CHECK	Authoritative

Design history. Two earlier designs were rejected: a shared spreadsheet on cloud storage (silent overwrites; sync conflicts break fixed-filename lookup) and a shared SQLite file on a network drive (network filesystems break SQLite locking, causing corruption). Both failed for want of a coordinating writer, not because of the storage engine. CON-3 prevents their reintroduction.

2.2 System Functions



System functions diagram: mess-proj decomposed into F1 Access control, F2 Room status, F3 Fault ticketing, F4 Live refresh, and F5 Reporting, with sub-functions under F1 (login/logout, role and mess scoping), F2 (view grid by floor, change status, manage room records), and F3 (log/filter/resolve)

F4 polls the server so every viewer sees changes within one interval; F5 aggregates across all messes for the org role.

2.3 User Characteristics

Role	Scope	Expertise	Frequency	Primary tasks
worker	Own mess	No software background; will not read a manual	Daily, short sessions	Change room status; log faults
secretary	Own mess	Administrative; no software background	Daily	Worker tasks + resolve tickets + manage rooms
org	All messes, read-only	Staff officer level	Weekly or ad hoc	Review cross-mess summary

2.4 Constraints, Assumptions, and Dependencies

Constraints

- **CON-1** The prototype shall use Python, Flask, Jinja2, Werkzeug, and SQLite via the standard sqlite3 module.
- **CON-2** The system shall use raw parameterized SQL; no ORM.
- **CON-3** Exactly one server process shall open the database for writing. Clients shall never access the database directly.
- **CON-4** A dependency shall be added only when the build phase in progress requires it.
- **CON-5** Security decisions shall never rely on the presentation tier. Hiding a control is a usability choice, not an access control.

Assumptions

- **ASM-1** User accounts are created administratively; there is no self-registration.
- **ASM-2** The prototype network is trusted at transport level.
- **ASM-3** Site, mess, and room identifiers are administratively stable.

- **ASM-4** Concurrent users per mess are low (single digits); read traffic dominates.

Dependencies

- **DEP-A** Rollout depends on DF information-systems approval (REG-5).
- **DEP-B** Rollout depends on infrastructure and network access being made available by the hosting organisation (REG-5).

2.5 Operational Environment

Aspect	Prototype	Rollout
Server OS	Developer workstation (Windows or Linux)	To be determined
Application server	Flask development server	To be determined
Database	SQLite file, WAL mode	To be determined
Client	Any current browser	To be determined
Network	Localhost or local network	To be determined
Real-time needs	None; 10–15 s polling	Same

3. Functional Requirements

Requirements are grouped by function (Section 2.2). Each group gives the template's descriptive fields, then the numbered requirements. Priority is MoSCoW for the prototype.

3.1 FR-1 Authentication and Session Management

Field	Content
Description	Establish who the user is and keep that identity across requests.
Inputs	Username, password (POST form); session cookie on later requests.
Processing	Look up user by username; verify password against stored hash; store user id in a signed session; on each request resolve the id to a user record.
Outputs	Redirect to dashboard on success; generic error on failure; redirect to login when unauthenticated.
Use cases	UC-1
Constraints	Session is signed, not encrypted. No authority at session level
Traceability	Build phase 4; [R4], [R5]

ID	Requirement	Priority
FR-1.1	The system shall store passwords only as salted hashes produced by Werkzeug's hashing helper; plaintext shall never be persisted or logged.	Must
FR-1.2	On successful login the system shall store only the user id in the session.	Must
FR-1.3	On failed login the system shall show one generic message that does	Must

ID	Requirement	Priority
	not reveal whether the username exists.	
FR-1.4	Logout shall accept POST only and shall clear the session.	Must
FR-1.5	Before any protected route runs, the system shall resolve the session to the user's id, role, and mess id.	Must
FR-1.6	An unauthenticated request to a protected route shall be redirected to the login page.	Must
FR-1.7	Password policy shall follow NIST SP 800-63B §5.1.1: minimum 8 characters, no forced composition rules, no forced periodic rotation.	Should

3.2 FR-2 Authorisation and Scoping

Field	Content
Description	Restrict every read and write to the messes and operations the user's role allows.
Inputs	Resolved user record (FR-1.5); target resource id; requested operation.
Processing	Scope resolver returns the mess constraint; the constraint is applied in the SQL WHERE clause; role checked against the operation.
Outputs	Scoped result set; or refusal with the status code fixed by FR-2.6.
Use cases	UC-2, UC-5
Constraints	CON-5. Scoping in application code after an out of scope fetch is prohibited.
Traceability	Build phase 5; Section 4.2

ID	Requirement	Priority
FR-2.1	Every query reading or writing rooms or tickets shall include the caller's scope in its SQL WHERE clause.	Must
FR-2.2	A single scope-resolver function shall return the caller's own mess id for worker and secretary, and no mess constraint for org. All scoped queries shall use it.	Must
FR-2.3	A request for a resource outside the caller's scope shall be refused before any write is attempted.	Must
FR-2.4	Creating, editing, and retiring room records shall be permitted to secretary only.	Must
FR-2.5	Every write route shall refuse the org role.	Must
FR-2.6	Out-of-scope requests shall return one consistent status code (403 or 404), selected and recorded under OI-1.	Must

3.3 FR-3 Room Status Management

Field	Content
Description	Show rooms as status tiles and let authorized users change a room's status.

Field	Content
Inputs	Floor (query parameter); room id and new status (POST); CSRF token.
Processing	Validate status; load room with scope in WHERE; UPDATE status, timestamp, actor; commit.
Outputs	Dashboard grid; redirect after write; flash message confirming success or failure.
Use cases	UC-2, UC-3
Constraints	Status set is fixed: available, occupied, maintenance.
Traceability	Build phases 3 and 5

ID	Requirement	Priority
FR-3.1	The dashboard shall show every in-scope room on the selected floor as a tile with room number, status, occupant, and last-updated time.	Must
FR-3.2	The floor shall be selectable by query parameter and filtered in SQL.	Must
FR-3.3	Each tile's status control shall be pre-selected to the room's current status.	Must
FR-3.4	A status change shall be rejected with 400 if the value is not in the permitted set, before the database is queried.	Must
FR-3.5	A successful status change shall set last_updated (ISO 8601) and last_updated_by and commit in one transaction.	Must
FR-3.6	A successful write shall respond with a redirect (POST/Redirect/GET).	Must
FR-3.7	Every write shall produce a flash message stating success or the reason for failure.	Should

3.4 FR-4 Fault Ticketing Lifecycle

Field	Content
Description	Record faults against rooms and track them to resolution.
Inputs	Room id, description (log); status filter (list); ticket id (resolve); CSRF token.
Processing	Scope-check the room; insert ticket stamped with reporter and ISO 8601 date; list via tickets → rooms → messes join; resolve stamps date_resolved.
Outputs	Ticket list; outstanding-count badge per tile; redirect and flash after writes.
Use cases	UC-3, UC-4
Constraints	Ticket fields are fixed columns; no generic attribute/value storage. State machine in Appendix D.
Traceability	Build phase 6

ID	Requirement	Priority
FR-4.1	A worker or secretary shall be able to log a ticket against any in-scope room, recording description, reported_by, and date_reported; status shall default to outstanding.	Must
FR-4.2	The ticket list shall be scoped by the scope resolver and ordered by date_reported, newest first.	Must
FR-4.3	The ticket list shall be filterable by outstanding, in progress, and resolved, filtered in SQL.	Must
FR-4.4	A secretary shall be able to move a ticket from outstanding to in progress.	Should
FR-4.5	Only a secretary shall be able to resolve a ticket; resolving shall set date_resolved.	Must
FR-4.6	Resolving an already-resolved ticket shall be refused.	Must
FR-4.7	Each room tile shall show its count of outstanding tickets.	Should
FR-4.8	Ticket descriptions shall be required and limited to 1,000 characters.	Should

3.5 FR-5 Live Polling Refresh

Field	Content
Description	Keep every open dashboard current without a manual reload.
Inputs	Floor number; session cookie.
Processing	Client requests JSON on a timer; server applies the scope resolver; client updates only changed tiles.
Outputs	JSON room state; updated tiles; last-synced indicator.
Use cases	UC-3
Constraints	Polling only; no websockets in the prototype.
Traceability	Build phase 7

ID	Requirement	Priority
FR-5.1	GET /api/rooms?floor=N shall return id, room number, status, occupant, and last-updated for in-scope rooms, applying the same scope resolver as page routes.	Must
FR-5.2	The client shall poll that endpoint every 10–15 seconds.	Must
FR-5.3	The client shall update changed classes and text only, never rebuild a tile's markup.	Must
FR-5.4	A tile containing the focused element shall be skipped in that cycle.	Must
FR-5.5	A network failure or non-OK HTTP status shall skip that cycle without stopping the polling loop.	Must
FR-5.6	The interface shall show the time of the last successful sync.	Should

3.6 FR-6 Aggregate Reporting

Field	Content
Description	Give the org role a read-only summary across all messes.
Inputs	Org user session.
Processing	SQL GROUP BY and aggregate queries over rooms and tickets.
Outputs	Summary page with per-site rollups.
Use cases	UC-5
Constraints	Read-only; aggregation done in SQL, not in Python or the browser.
Traceability	Build phase 8

ID	Requirement	Priority
FR-6.1	The org role shall see rooms by status, per mess.	Must
FR-6.2	The org role shall see outstanding tickets per site.	Must
FR-6.3	The org role shall see mean time to resolution, computed over resolved tickets only.	Should
FR-6.4	Aggregates shall be computed in SQL.	Must
FR-6.5	Indexes beyond those in DAT-2 shall be added only when a query plan shows a full-table scan, and each shall be recorded under OI-3.	Should

4. Non-Functional Requirements

4.1 Performance Requirements

ID	Requirement	Priority
NFR-PERF-1	A status change made in one browser shall appear in every other in-scope browser within one polling interval (≤ 15 s).	Must
NFR-PERF-2	Page and JSON responses shall complete in under 500 ms on the prototype host with seed data.	Should
NFR-PERF-3	Aggregate queries (FR-6) shall use indexes rather than full-table scans, as shown by EXPLAIN QUERY PLAN.	Should
NFR-PERF-4	The server shall close each request's database connection at teardown; connection count shall not grow under sustained polling.	Must

Rollout latency targets are deferred until prototype measurements exist (OI-5).

4.2 Safety and Security Requirements

ID	Requirement	Priority
NFR-SEC-1	All SQL shall use bound parameters; string interpolation into SQL is prohibited.	Must
NFR-SEC-2	Jinja2 autoescaping shall stay enabled for every template rendering user-supplied data.	Must

ID	Requirement	Priority
NFR-SEC-3	Every state-changing request shall carry a CSRF token verified server-side (implementation per OI-2).	Must
NFR-SEC-4	The session signing key shall be read from the environment; in production mode, a missing key shall stop startup rather than fall back.	Must
NFR-SEC-5	Every authorization and validation check shall run on the server, regardless of interface state (CON-5).	Must
NFR-SEC-6	Session cookies shall be HttpOnly and SameSite=Lax; Secure shall be set once served over HTTPS.	Should

Threat model

Threat	Mitigation	Requirements
Cross-mess data leakage	Scope in SQL WHERE	FR-2.1, FR-2.2
SQL injection	Parameterised queries	NFR-SEC-1
Cross-site scripting	Autoescaping	NFR-SEC-2
Cross-site request forgery	Per-form token; POST-only logout	NFR-SEC-3, FR-1.4
Session forgery or theft	Signed cookie; env-sourced key; cookie flags	FR-1.2, NFR-SEC-4, NFR-SEC-6
Credential disclosure	Hashing; generic login errors	FR-1.1, FR-1.3
Trusting the client	Server-side checks only	NFR-SEC-5

4.3 Reliability and Availability

ID	Requirement	Priority
NFR-REL-1	Every connection shall issue PRAGMA foreign_keys = ON; the database shall run in WAL mode.	Must
NFR-REL-2	Every data-modifying request shall commit explicitly or roll back; no write shall be left in an implicit, uncommitted transaction.	Must
NFR-REL-3	A transient client network failure shall not stop the dashboard from recovering on the next cycle (FR-5.5).	Must
NFR-REL-4	Unhandled server errors shall return a generic 500 page without stack traces when not in debug mode.	Must
NFR-REL-5	Rollout availability target: service during standard organisational working hours, with a documented restore-from-backup procedure (DEP-5).	Should

4.4 Usability and Human Factors

ID	Requirement	Priority
NFR-USE-	A first-time worker shall be able to change a room's status and log	Must

ID	Requirement	Priority
1	a fault without instruction.	
NFR-USE-2	Status shall be shown by text as well as colour.	Must
NFR-USE-3	Tile colours shall be defined only in the stylesheet via state classes (tile-available, etc.).	Should
NFR-USE-4	Text and status colours shall meet WCAG 2.1 AA contrast (4.5:1).	Should
NFR-USE-5	The navigation strip shall always show username, role, and a logout control.	Should

4.5 Maintainability and Support

ID	Requirement	Priority
NFR-MNT-1	Code shall follow the project layout in [R1] (app.py, db.py, schema.sql, seed.py, templates/, static/, tests/).	Must
NFR-MNT-2	Every decision with more than one defensible option shall be recorded in the README decision log when made.	Must
NFR-MNT-3	Commits shall be small and descriptively messaged.	Should
NFR-MNT-4	Runtime and development dependencies shall be listed separately (requirements.txt, requirements-dev.txt).	Should
NFR-MNT-5	The database shall be rebuildable from schema.sql and seed.py with one command.	Must

4.6 Portability and Compatibility

ID	Requirement	Priority
NFR-PRT-1	Database access shall use only the PEP 249 interface.	Must
NFR-PRT-2	Dates shall be ISO 8601 throughout.	Must
NFR-PRT-3	The interface shall work in current Chromium-based browsers and Firefox without plugins.	Must
NFR-PRT-4	The application shall run on Windows and Linux.	Should

4.7 Other Quality Attributes

ID	Requirement	Priority
NFR-QA-1 (testability)	Tests shall run against a fresh seeded database per test.	Must
NFR-QA-2 (testability)	The authorisation matrix shall be covered by parametrised tests: role × scope × operation, for page and JSON routes.	Must
NFR-QA-3	Every room row shall record who last changed it and when.	Must

ID	Requirement	Priority
(auditability)		

5. External Interface Requirements

5.1 User Interfaces

All pages extend base.html, which provides the head, stylesheet, navigation strip, flash region, and a content block. There is no command-line interface for end users.

ID	Screen	Route	Contents
IF-1	Login	GET/POST /login	Username, password, submit; generic error area
IF-2	Dashboard	GET /?floor=N	Floor toggle; CSS grid of room tiles (number, status, occupant, last updated, ticket badge, status control, log-fault link)
IF-3	Tickets	GET /tickets? status=S	Status filter; table of room, description, reporter, dates, status; resolve control for secretary
IF-4	Room management	/rooms/... (secretary)	Create, edit, retire room forms
IF-5	Summary	GET /summary (org)	Rooms by status per mess; outstanding tickets per site; mean time to resolution

An operator-only command (IF-6: flask init-db) shall drop and recreate the database from schema.sql and seed it.

5.2 Hardware Interfaces

No special hardware. Clients are existing PCs with a browser and a display of at least 1280×720. The server needs no peripherals.

5.3 Software Interfaces

ID	Interface	Direction	Format	Contract
IF-7	GET /api/rooms?floor=N	Server → browser	JSON array	[{"id": int, "room_number": str, "status": "available" "occupied" "maintenance", "occupant": str null, "last_updated": ISO 8601 str null}]; 401 if unauthenticated; 400 on malformed floor
IF-8	POST /rooms/<id>/status	Browser → server	Form-encoded: status,	302 on success; 400 invalid value; FR-2.6 code if out of scope

ID	Interface	Direction	Format	Contract
IF-9	POST /tickets	Browser → server	csrf_token Form- encoded: room_id, description, csrf_token	302 on success; 400 invalid; FR-2.6 code if out of scope
IF-10	POST /tickets/<id>/resolve	Browser → server	Form- encoded: csrf_token	302; 409 if already resolved
IF-11	Database driver	Server → DB	PEP 249	sqlite3 (prototype), psycopg (rollout)

Library	Role	Release
Flask	Routing, sessions, app context	Both
Jinja2	Templating, autoescaping	Both
Werkzeug	Password hashing, WSGI utilities	Both
flask-wtf	CSRF protection	From phase 5
pytest	Test runner (dev only)	From phase 9
Gunicorn, nginx	WSGI server, reverse proxy	Rollout

5.4 Communications Interfaces

ID	Requirement	Priority
IF-12	prototype on the Flask default port 5000.	Must
IF-13	Rollout traffic shall use HTTPS (TLS 1.2 or later); HTTP shall redirect to HTTPS.	Must (rollout)
IF-14	Polling bandwidth shall stay under 10 KB per client per cycle for a 40-room floor.	Could
IF-15	The database shall not be reachable over the network in the prototype.	Must

6. Other Requirements

6.1 Legal and Regulatory Requirements

Occupant names tied to rooms, and reporters named on tickets, are personal data under GDPR [R6] and the Data Protection Act 2018 [R7].

ID	Requirement	Source	How compliance is assured
REG-1	Collect only fields needed to run the mess: no personal data beyond occupant name and user	GDPR Art. 5(1)(c), Art.	Schema review against Section

ID	Requirement	Source	How compliance is assured
	identity.	25	6.4
REG-2	Define a lawful basis for processing before rollout.	GDPR Art. 6	Recorded by DF data controller
REG-3	Define and implement a retention period for occupant and ticket data before rollout.	GDPR Art. 5(1)(e); DPC guidance	OI-4; schema change if needed
REG-4	Apply security of processing appropriate to the risk.	GDPR Art. 32	Section 4.2 controls; accreditation
REG-5	Obtain the appropriate information-systems approval before any deployment on organisational infrastructure, and before writing rollout-targeted code.	Internal policy (non-public)	Written approval on file
REG-6	Maintain a record of processing activities once deployed.	GDPR Art. 30	DF data protection office
REG-7	Before the repository is public: generic naming, neutral site names, organisation vocabulary removed from code, comments, commits and branches, fictional seed data, and git history checked.	Project policy	Pre-publication checklist in [R1] Appendix C

6.2 Packaging, Installation, and Deployment

ID	Requirement	Priority
DEP-1	A fresh clone shall install with <code>pip install -r requirements.txt</code> in a new virtual environment and run with documented commands.	Must
DEP-2	The database file, virtual environment, caches, and environment files shall be git-ignored.	Must
DEP-3	Rollout shall run behind a production WSGI server and reverse proxy, with HTTPS and an environment-sourced secret key.	Must (rollout)
DEP-4	The application and database shall be startable with a single command via containers.	Could (portfolio P4)
DEP-5	Rollout shall include scheduled database backups and a tested restore procedure.	Must (rollout)
DEP-6	A public demo, if deployed, shall contain fictional data only.	Must

6.3 Internationalisation and Localisation

ID	Requirement	Priority
LOC-1	The interface language shall be English (Ireland spelling).	Must
LOC-2	Timestamps shall be stored as ISO 8601 in UTC and displayed in	Should

ID	Requirement	Priority
LOC-3	Europe/Dublin local time as DD Mon YYYY HH:MM. Text columns shall accept UTF-8, including Irish-language fadas in names.	Must

6.4 Data Requirements

ID	Requirement	Priority
DAT-1	The schema shall be exactly the DDL below; any change shall be made in this section first.	Must
DAT-2	Indexes idx_rooms_mess and idx_tickets_room shall exist from the start; others per FR-6.5.	Must
DAT-3	Domain values shall be enforced by CHECK constraints and also validated in application code.	Must
DAT-4	Foreign keys shall be enforced on every connection (NFR-REL-1).	Must
DAT-5	Dates shall be stored as ISO 8601 TEXT so lexical and chronological order agree.	Must
DAT-6	Rooms shall be retired, not deleted, while tickets reference them.	Should

An entity-relationship diagram is in Appendix D.

```
CREATE TABLE sites (
id INTEGER PRIMARY KEY,
name TEXT NOT NULL UNIQUE
);
```

```
CREATE TABLE messes (
id INTEGER PRIMARY KEY,
site_id INTEGER NOT NULL REFERENCES sites(id),
mess_type TEXT NOT NULL CHECK (mess_type IN ('a','b','c')),
UNIQUE (site_id, mess_type)
);
```

```
CREATE TABLE users (
id INTEGER PRIMARY KEY,
username TEXT NOT NULL UNIQUE,
password_hash TEXT NOT NULL,
role TEXT NOT NULL CHECK (role IN ('worker','secretary','org')),
mess_id INTEGER REFERENCES messes(id) – NULL for org role
);
```

```
CREATE TABLE rooms (
id INTEGER PRIMARY KEY,
mess_id INTEGER NOT NULL REFERENCES messes(id),
room_number TEXT NOT NULL,
floor INTEGER NOT NULL,
status TEXT NOT NULL DEFAULT 'available'
```

```
CHECK (status IN ('available','occupied','maintenance')),
occupant TEXT,
last_updated TEXT, – ISO 8601
last_updated_by INTEGER REFERENCES users(id),
UNIQUE (mess_id, room_number)
);
```

```
CREATE TABLE tickets (
id INTEGER PRIMARY KEY,
room_id INTEGER NOT NULL REFERENCES rooms(id),
description TEXT NOT NULL,
status TEXT NOT NULL DEFAULT 'outstanding'
CHECK (status IN ('outstanding','in_progress','resolved')),
date_reported TEXT NOT NULL, – ISO 8601
date_resolved TEXT,
reported_by INTEGER REFERENCES users(id)
);
```

```
CREATE INDEX idx_rooms_mess ON rooms (mess_id);
CREATE INDEX idx_tickets_room ON tickets (room_id, status);
```

Connection-time settings, issued on every connection rather than in the schema file:

```
PRAGMA foreign_keys = ON; – per connection; off by default
PRAGMA journal_mode = WAL; – persists in the database file
```

Declaration	Rationale
INTEGER PRIMARY KEY	Aliases rowid; AUTOINCREMENT not needed as id reuse after deletion is acceptable
Nullable users.mess_id	Org users have no single mess; NULL means “not applicable”
UNIQUE (site_id, mess_type)	One mess of each type per site; neither column unique alone
UNIQUE (mess_id, room_number)	Room numbers repeat across messes, not within one
DEFAULT 'available'	Seeding can omit status without breaking NOT NULL

Note on DAT-6: the schema has no retired flag yet. Satisfying DAT-6 needs either a retired column or a fourth room status; tracked as OI-6.

7. Verification and Validation

7.1 Requirements Verification Methods

Four methods apply: **Test** (automated pytest), **Demonstration** (manual run, observed), **Inspection** (reading code, schema, or data), **Analysis** (query plans, measurements). Every test shall be seen to fail before it is accepted as passing.

Requirement	Method	Verification
FR-1.1	I, T	Inspect users table directly: hashes only. Test: login with seeded password succeeds
FR-1.2 – FR-1.6	T	test_auth.py: login success/failure, identical failure messages, POST-only logout, redirect when logged out
FR-1.7	T	Short password rejected; 8+ with no composition accepted
FR-2.1, FR-2.2	I, T	Inspect every rooms/tickets query for resolver use; parametrised matrix in test_rooms.py
FR-2.3 – FR-2.6	T, D	Matrix tests; hand-crafted cross-mess request from command line (curl) returns the OI-1 code
FR-3.1 – FR-3.3	D	Grid, floor toggle, and pre-selected controls observed in browser
FR-3.4 – FR-3.6	T	Invalid status → 400; valid → 302 and row updated with timestamp and actor
FR-3.7, FR-3.8	T, D	Flash shown; secretary room CRUD allowed, worker refused
FR-4.1 – FR-4.8	T	test_tickets.py: scoping, ordering, filtering, resolve permissions, re-resolve refused, length limit
FR-5.1	T	JSON shape; cross-mess and unauthenticated calls refused
FR-5.2 – FR-5.6	D	Two-window test; open dropdown survives refresh; server stopped → indicator stalls, loop recovers on restart
FR-6.1 – FR-6.4	T, I	Aggregates match hand-counted rows for seed data
FR-6.5, NFR-PERF-3	A	EXPLAIN QUERY PLAN shows index use; justification logged
NFR-PERF-1, 2, 4	A, D	Timed requests; connection count stable after 10 min polling
NFR-SEC-1	I, D	Grep for string-built SQL; injection exercise on throwaway branch
NFR-SEC-2 – 6	T, I	Script-tag occupant renders inert; missing CSRF → 400; startup fails without key in production; cookie flags inspected
NFR-REL-1, 2	T	Bad foreign key insert rejected; write committed and visible on new connection
NFR-REL-3, 4	D	Network drop recovery; 500 page has no traceback
NFR-USE-1	D	Unassisted first-use walkthrough with a mess staff member
NFR-USE-2 – 5	I	Stylesheet and template review; contrast checker
NFR-MNT,	I, D	Layout, decision log, commit history review; clean-clone install

Requirement	Method	Verification
NFR-PRT		on Windows and Linux
NFR-QA-1 – 3	I, T	Fixture review; matrix coverage count; last_updated_by populated
IF-1 – IF-15	T, D	Route and contract tests; manual screen review
REG-1 – REG-7	I	Schema and history review; approval documents on file
DEP, LOC, DAT	I, D	Clean-clone install; timestamp display; fada round-trip; constraint violation attempts

The full requirement-to-phase-to-test mapping is Appendix C.

7.2 Acceptance Criteria

Each build phase is accepted only when its criterion is run and observed, not inferred.

Phase	Acceptance criterion
0 Skeleton	≥ 4 small descriptive commits; git status clean with venv present; clean clone installs and imports Flask
1 Data layer	Seeded rows queryable by column name; bad foreign key, bad mess_type, duplicate room each rejected; tickets sort chronologically
2 First route	Browser shows live database rows; connections close per request
3 Interface	Colour-coded grid renders; floor toggle works; each control shows true status; nothing writes
4 Authentication	Logged-out dashboard redirects; wrong password rejected; users shows hashes only
5 Authorisation	Hand-crafted cross-mess write refused with the OI-1 code
6 Ticketing	Log – filter → resolve works end to end; dates sort correctly
7 Polling	Change syncs across two windows within 15 s; open dropdown survives; JSON refuses cross-mess
8 Reporting	One aggregate verified by hand; query plan shows index use
9 Testing	All tests pass; each observed failing first; prototype tagged
Prototype release	All Must requirements verified; no open Must-level defects
Rollout release	Prototype accepted + REG-2, 3, 5, 6 closed + DEP-3, DEP-5 verified + NFR-USE-1 validated with real mess staff

8. Appendices

Appendix A: Glossary

Term	Definition
Aggregate view	The org-only summary page (FR-6)

Term	Definition
Decision log	README section recording constraint, options, choice, and trade-off for each decision
Focus guard	Client rule that skips refreshing a tile holding keyboard focus (FR-5.4)
Implicit transaction	Transaction opened automatically by sqlite3 on the first write; lost unless committed
POST/Redirect/GET	Responding to a write with a redirect so a reload does not resubmit it
Query plan	SQLite's EXPLAIN QUERY PLAN output showing index or scan use
Row factory	sqlite3 setting that makes rows addressable by column name
Scope resolver	Function returning the mess constraint for the current user (FR-2.2)

Appendix B: Use Cases

ID	Actor	Goal	Main flow	Alternate flows
UC-1	Any user	Sign in	Open site → redirected to login → enter credentials → dashboard	Wrong password: generic error, stay on login
UC-2	Worker	Mark a room occupied	Dashboard → choose floor → set tile to occupied → submit → redirect with success flash; other screens update within 15 s	Room in another mess via crafted request: refused (FR-2.6)
UC-3	Worker	Report a leaking tap	Tile → log fault → enter description → submit → ticket listed as outstanding; tile badge increments	Empty description: 400 with message
UC-4	Secretary	Close a fixed fault	Tickets → filter outstanding → resolve → ticket moves to resolved with date	Already resolved: refused
UC-5	Org user	Review brigade status	Login → summary page → per-mess room counts, outstanding tickets per site, mean resolution time	Attempts a write: refused (FR-2.5)

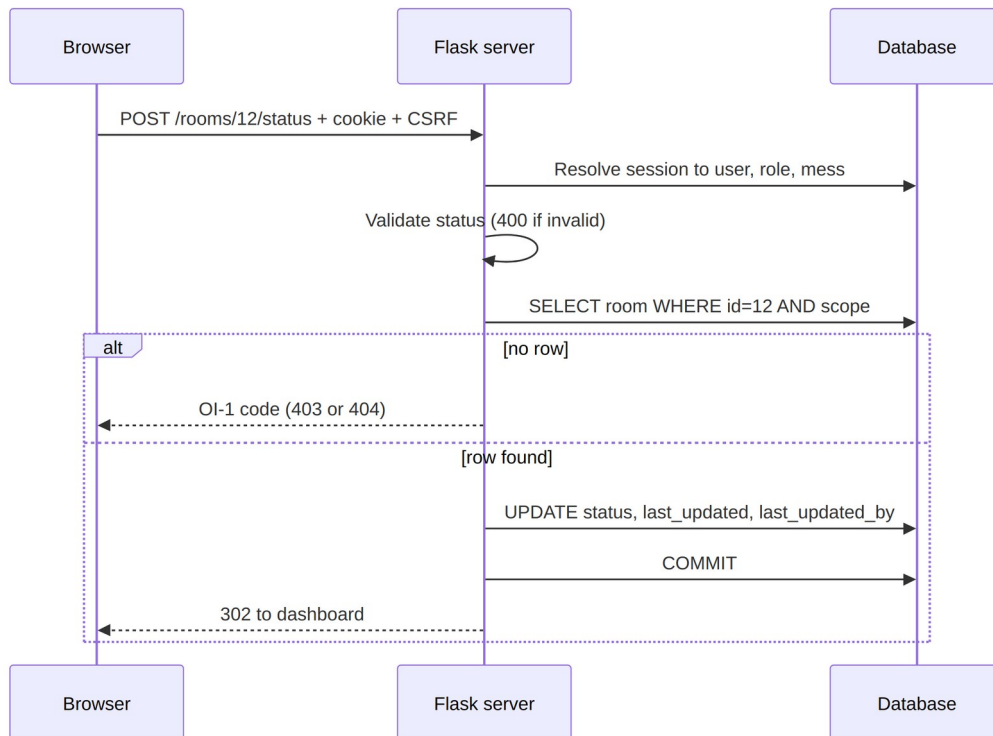
Appendix C: Traceability Matrix

Requirements	Source	Build phase	Design element	Test / evidence
CON-1–5, NFR-MNT-1, 3, DEP-1, 2	[R1] Phase 0; [R2] \$1–2	0	Repo, layout, .gitignore	Commit log; clean clone
DAT-1–6, NFR-REL-1, NFR-MNT-5	[R2] \$4, \$15	1	schema.sql, db.py, seed.py	Constraint-violation attempts
NFR-PERF-4	[R1]	2	db.py teardown	Connection count check

Requirements	Source	Build phase	Design element	Test / evidence
	2.3			
FR-3.1–3.3, NFR-USE-2–4, NFR-SEC-2	[R2] §3 Phase 3	3	base.html, dashboard.html, style.css	Browser demonstration
FR-1.1–1.7, NFR- SEC-4, 6	[R2] Phase 4; [R4]; [R5]	4	login route, before-request hook, login_required	test_auth.py
FR-2.1–2.6, FR- 3.4–3.8, NFR- SEC-1, 3, 5, NFR- REL-2	[R2] §5, Phase 5; [R4]	5	scope resolver, write routes, flask- wtf	test_rooms.py matrix; curl test
FR-4.1–4.8	[R2] Phase 6	6	tickets routes, tickets.html	test_tickets.py
FR-5.1–5.6, NFR-PERF-1, NFR-REL-3	[R2] Phase 7	7	/api/rooms, app.js	Two-window demo; JSON tests
FR-6.1–6.5, NFR-PERF-3	[R2] Phase 8	8	summary route, indexes	Hand count; query plan
NFR-QA-1–3	[R2] Phase 9	9	conftest.py fixtures	Suite run; fail-first log
NFR-PRT-1, 2	[R1] P2	Portfolio P2	PEP 249 usage	Migration diff
DEP-3–6, IF-13	[R1] P4, P5	Portfolio P4–P5	Gunicorn, nginx, containers	Deployment demo
REG-1–7	[R2] §14; [R6]; [R7]	Pre-publication / pre-rollout	Schema, repo history	Checklist and approvals

Appendix D: Supporting Diagrams

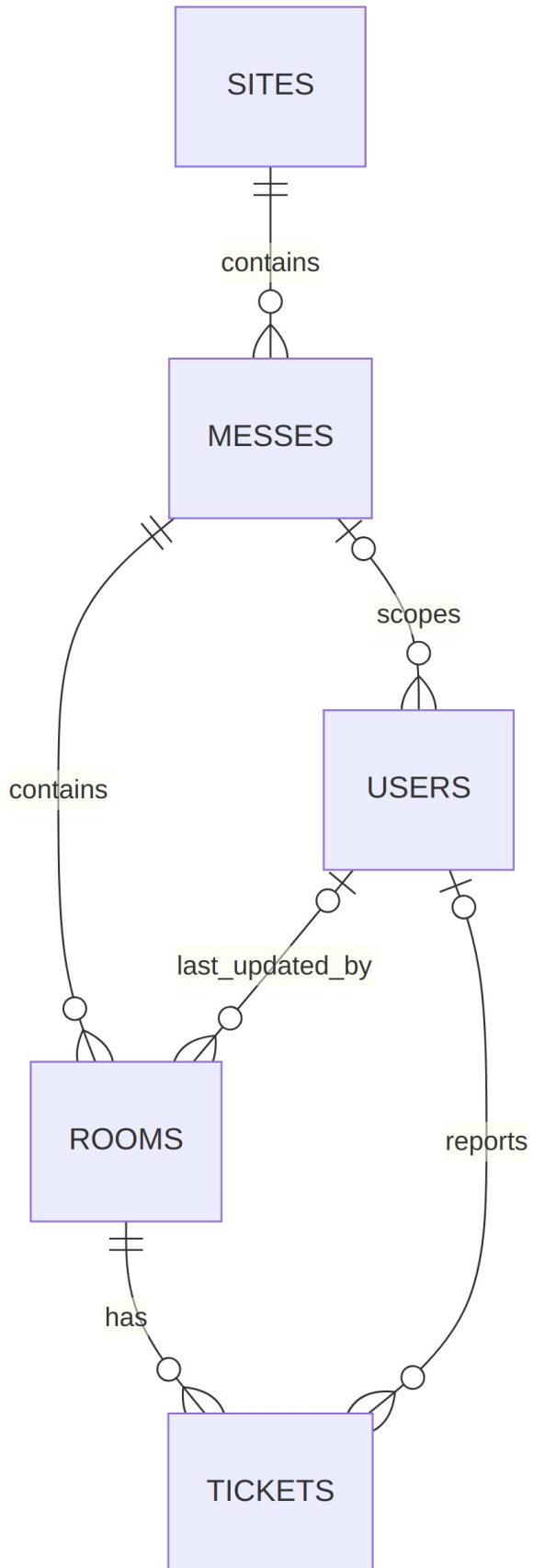
D.1 Request lifecycle (status change)



Sequence diagram of the status-change request lifecycle: browser posts a status change, the server resolves the session, validates the status, checks scope, and either refuses out-of-scope requests or updates and commits the change

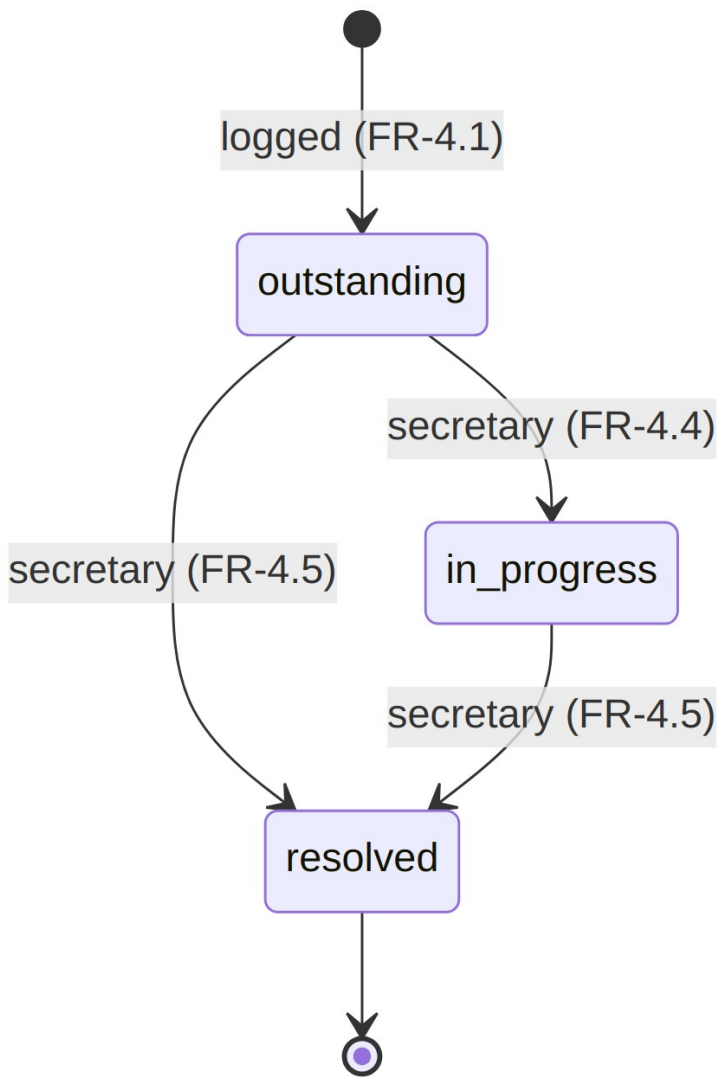
Validation runs before the scoped load because it is cheaper; the scope check is the step most often skipped.

D.2 Entity relationships



Entity relationship diagram: Sites contain Messes, Messes contain Rooms and scope Users, Rooms have Tickets, and Users report Tickets and are recorded as last_updated_by on Rooms

D.3 Ticket states



Ticket state diagram: a new ticket starts outstanding, a secretary can move it to in progress or directly to resolved, and in-progress tickets can be resolved; resolved is terminal

Resolved is terminal; re-resolution is refused (FR-4.6).

Appendix E: Open Issues (TBD)

Each must be closed in the README decision log before the dependent phase is accepted.

ID	Issue	Options	Blocks
OI-1	Status code for out-of-scope	403 (reveals existence) or 404 (conceals	Phase 5

ID	Issue	Options	Blocks
	resources	it)	
OI-2	CSRF implementation	Hand-rolled on throwaway branch for learning; flask-wtf shipped	Phase 5
OI-3	Index additions beyond DAT-2	Only on query-plan evidence	Phase 8
OI-4	Retention period for occupant and ticket data	To follow GDPR Art. 5 review with DF data protection office	Rollout
OI-5	Rollout latency and availability targets	Set from prototype measurements	Rollout
OI-6	How rooms are retired (DAT-6)	retired boolean column, or a fourth status value	Phase 5 (FR-3.8)

Appendix F: Revision History

Version	Date	Author	Change
1.0	2026-09-24	Zyad	Initial free-form SRS
2.0	2026-09-24	Zyad	Restructured to ISO/IEC/IEEE 29148 outline; added per-requirement priority, verification methods, use cases, traceability matrix, diagrams, communications, localisation, and deployment requirements
2.1	2026-09-24	Zyad	Sanitisation pass: removed named sites, unit designation, and any network/infrastructure specifics ahead of publication (REG-7); diagrams re-rendered as images